

MACHINE LEARNING FUNDAMENTALS SERIES

Neural Networks Pt. 3: ReLU In Action!!!

Piecewise Linear Activation, Shape Synthesis, and Non-Linear Decision Boundaries

Source: StatQuest with Josh Starmer — Neural Networks Pt. 3: ReLU In Action!!!

Crucial Pedagogical Context

Like the previous Softplus demonstration, this network uses a **minimalist pre-trained architecture** (1 Input, 1 Hidden Layer with 2 Nodes, 1 Output with ReLU). All weights and biases are **already optimized** via Backpropagation. The purpose of this lesson is to demonstrate **how ReLU transforms straight lines into bent shapes** ("green pointy things") to fit complex non-linear data.

1. Introduction: Softplus vs. ReLU Activation

In Part 1, we saw how a neural network using the smooth **Softplus** activation function ($\text{Softplus}(x) = \ln(1 + e^x)$) fits a smooth "green squiggle" to a drug dosage dataset.

While Softplus creates smooth curves, modern Deep Learning and Convolutional Neural Networks (CNNs) predominantly rely on the **ReLU (Rectified Linear Unit)** activation function due to its extreme computational efficiency and superior gradient propagation.

The ReLU Function Definition:

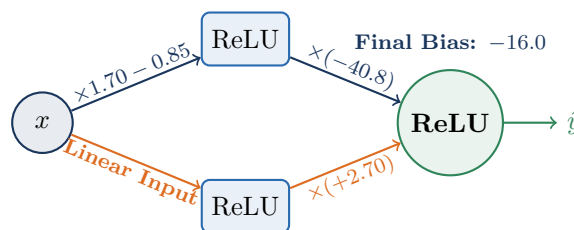
$$\text{ReLU}(x) = \max(0, x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

2. Network Architecture & Parameter Setup

Consider the binary classification task: predicting whether a drug dosage $x \in [0.0, 1.0]$ is effective ($y = 1.0$) or ineffective ($y = 0.0$):

- **Low Dosage** ($x = 0.0$): Ineffective ($y = 0.0$).
- **Medium Dosage** ($x = 0.5$): Highly Effective ($y = 1.0$).
- **High Dosage** ($x = 1.0$): Toxic / Ineffective ($y = 0.0$).

Linear models fit a straight line and can at most predict 2 out of 3 points. Below is the pre-trained neural network architecture using ReLU at both the hidden layer and output layer:



3. Step-by-Step Mathematical Line Transformation

3.1. Top Hidden Node (Bent Blue Line)

1. **Linear Transformation:** $x_{\text{top}} = 1.70 \cdot x - 0.85$.
2. **ReLU Activation:** $y_{\text{top}} = \text{ReLU}(1.70x - 0.85)$.
 - For $x \leq 0.5$ (e.g., $x = 0.0, 0.2, 0.4$), $x_{\text{top}} \leq 0 \implies \text{ReLU} = 0.0$.
 - For $x > 0.5$ (e.g., $x = 0.6 \implies x_{\text{top}} = 0.17$), $\text{ReLU} = 0.17$.

This produces a **bent blue line** flat at 0 for $x \leq 0.5$, then sloping upward for $x > 0.5$.

3. **Output Weight Multiplication:** Multiply by -40.8 :

$$\text{Line}_{\text{blue}} = -40.8 \cdot \text{ReLU}(1.70x - 0.85)$$

Multiplying by a negative weight **flips the bent line downward** and stretches it steeply.

3.2. Bottom Hidden Node (Straight Orange Line)

The bottom node processes the dosage into a straight line and multiplies it by +2.70, producing a **straight orange line** with a positive slope.

3.3. Output Node Aggregation & Output ReLU

- Summation of Branches:** Adding the bent blue line and the straight orange line creates a **Green Wedge** (a V-shaped/bent line).
- Final Bias Subtraction:** Subtracting the final bias -16.0 shifts the entire green wedge downward.
- Output ReLU Filter:** Passing through the output ReLU sets all negative values to 0.0 while keeping positive values unchanged:

$$\hat{y} = \text{ReLU}\left(\text{Line}_{\text{blue}} + \text{Line}_{\text{orange}} - 16.0\right)$$

This transforms the shifted green wedge into a **”Green Pointy Thing”** (a sharp trapezoidal peak) that equals 1.0 at Medium Dosage ($x = 0.5$) and drops to 0.0 at Low ($x = 0.0$) and High ($x = 1.0$) dosages.

4. Technical Alert: Non-Differentiability at $x = 0$

Gradient Descent requires computing the derivative $\frac{d}{dx}\text{ReLU}(x)$ at every point. However, at $x = 0$, the function has a sharp bend (kink) and is **mathematically non-differentiable**:

Subgradient Descent in Practice

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{Undefined} & \text{if } x = 0 \end{cases}$$

Practical Solution in Deep Learning Frameworks:

Machine learning libraries handle this by explicitly defining the derivative at $x = 0$ to a fixed value (typically 0 or 1). Because floating-point inputs almost never land exactly on 0.00000000, subgradient descent converges reliably without numerical instability.

5. Summary Comparison: Activation Functions

Table 1: Comparison of Popular Neural Network Activation Functions

Feature / Property	ReLU	Softplus	Sigmoid
Formula	$\max(0, x)$	$\ln(1 + e^x)$	$\frac{1}{1+e^{-x}}$
Output Shape	Piecewise Linear / Bent	Smooth Curve	Smooth S-Curve
Output Range	$[0, \infty)$	$(0, \infty)$	$(0, 1)$
Computation Cost	Extremely Low (Thresholding)	High (ln, exp)	High (exp)
Gradient Behavior	Constant (1 for $x > 0$)	Smooth	Vanishing Gradient for $ x \gg 0$
Best Use Case	Hidden Layers in Deep Learning & CNNs	Smooth Function Fitting	Output Layer for Binary Classification

Core Takeaway

Despite its extreme simplicity, the ReLU activation function enables neural networks to fit complex non-linear boundaries. By slicing, flipping, and stretching bent lines and combining them through layers, networks build piecewise linear approximations (”pointy shapes”) that generalize exceptionally well while remaining computationally fast.